

Speculating About Store-Buffer Capacity

2018-04-07 20:33:45

Original: <https://nicknash.me/2018/04/07/speculating-about-store-buffer-capacity/>

1 Removing Fences

If you're ever looking to squeeze the performance of lock-free code it's tempting to try and get rid of unnecessary memory fences. On x86/x64, this can be tantalising, as there's only a single type of memory fence required at the processor level: the store-load fence.

So on such architectures the question arises: Are there thread-safe data structures where we can do without even a store-load fence?

For coordinating just two threads, the answer is of course yes, as only acquire-release semantics are required for example by a SPSC queue. But what about more generally?

The answer turns out to also be yes, as shown in this ASPLOS 2014 paper: [Fence-Free Work Stealing on Bounded TSO Processors](#)

Doing this requires guessing the "re-order bound" of the processor in question, which is more-or-less just the capacity of its *store buffer*. In this post, I'm going to describe and reproduce an experiment the paper uses that allows guessing the store buffer capacity of an Intel processor.

2 A Tiny Bit of Background

So what are store buffers and why do processors have them? One reason is that processors are typically *speculative*, meaning that they will execute instructions before it is certain those instructions are actually required. This is enabled by so-called *branch prediction*, which roughly amounts to the processor guessing ahead of time which way an if-statement will go.

One type of instruction that makes speculation tricky is a store to memory. If the store in question turns out not to be required after all, then the processor would need to somehow undo the store in memory. That would be bad enough, but by speculatively storing to memory the visibility of the store to other processors would make programming such a machine hell. The solution to this is to have stores first write into the store buffer, and then only later "drain" the values from the store buffer to cache when the branch prediction has been verified and the other required conditions are met.

3 Store Buffers Meet Memory Models

In the x86 memory model, it's the store-buffer then that introduces the possibility for an earlier (in program order) store to be re-ordered with a subsequent load. This famously makes Petersen's mutex (or "Dekker's algorithm") incorrect without a store-load fence. A minimal demonstration from Intel's memory ordering white paper is [here](#).

Stores can't be re-ordered past an arbitrary number of loads though - store buffer capacity is finite. In the *Intel® 64 and IA-32 Architectures Optimization Reference Manual* they report the store buffer as having 42 entries for a Haswell microarchitecture, for example.

4 An Experiment

Enough background! How can we indirectly observe the store buffer capacity? Well, Intel includes this warning in the *Intel® 64 and IA-32 Architectures Optimization Reference Manual*:

As long as the store does not complete, its entry remains occupied in the store buffer. When the store buffer becomes full, new micro-ops cannot enter the execution pipe and execution might stall.

So to get a glimpse of the store buffer in action, the start of the idea is to repeatedly execute a sequence of *S* stores, varying *S* from say 1 up to 100, and computing the average time per iteration, something like this:

```
for S = 1 to 100
  Record StartTime
  for N = 1 to 100000 # Just repeat a fair few times
    Perform S stores
  Record EndTime
  Report (EndTime - StartTime)/100000 # Avg time per iteration
```

Now, this won't give a very interesting result:

Missing diagram.

Alt: Screen Shot 2018-04-04 at 17.26.31

Source: <https://nicknash.me/wp-content/uploads/2018/04/screen-shot-2018-04-04-at-17-26-31.png>

This isn't giving much away about the store-buffer. The reason is that there is no time for the store buffer to drain, so all the iterations end up suffering from store-buffer stalls.

A simple way to give the store buffer time to drain is to just execute a stream of other instructions after them, even a simple no-op will do:

```
for S = 1 to 100
  Record StartTime
  for N = 1 to 100000 # Just repeat a fair few times
    Perform S stores
    Perform 500 no-ops # Allow store buffer to drain
  Record EndTime
  Report (EndTime - StartTime)/100000 # Avg time per iteration
```

What now happens is quite interesting: while there are remaining store-buffer entries, the latency of the no-ops will be at least partially hidden. Once the stores have been pushed to execution ports,

the first few hundred no-ops can then "execute" from the re-order buffer (no-ops don't really execute, and are eliminated in the "Renamer" portion of the pipeline, but the processor can only eliminate 4 of them per cycle so they do consume some time) until the stores retire and their store buffer entries are freed.

But what about when we attempt a store where no store buffer entry is available? To get a read on that we can again refer to our trusty *Intel® 64 and IA-32 Architectures Optimization Reference Manual*:

Firstly, the description of the Renamer includes the following:

The Renamer is the bridge between the in-order part [...] and the dataflow world of the Scheduler. It moves up to four micro-ops every cycle from the micro-op queue to the out-of-order engine.

As well as performing register renaming, we're told that the Renamer:

Allocates resources to the micro-ops. For example, load or store buffers.

So it really *seems* that if a store buffer entry isn't available, the store in question won't proceed past renaming, and more importantly, the subsequent no-ops won't be executed until it does so. As a result, however much of the latency of the stream of no-ops was previously hidden should become visible, as they no longer execute in parallel with the stores.

On my Haswell, this is the result I see:

Missing diagram.

Alt: Screen Shot 2018-04-04 at 20.47.45

Source: <https://nicknash.me/wp-content/uploads/2018/04/screen-shot-2018-04-04-at-20-47-45.png>

The jump upwards in latency appears when 43 stores are attempted, and Haswell's documented store buffer capacity is 42. The steepening is then (I guess) because each subsequent store must wait for a store buffer entry to become free.

So that's it! We've revealed the 42 entry store buffer. Of course, nothing is stopping us from measuring a bit more directly.

5 Measuring

To get some independent evidence that this behaviour is a result of stalls caused by the store buffer, we can crank out VTune. The relevant event is RESOURCE_STALLS.SB, which Intel document as:

This event counts the number of cycles that a resource related stall will occur due to the number of store instructions reaching the limit of the pipeline, (for example, all store buffers are used). The stall ends when a store instruction commits its data to the cache or memory.

For 42 stores on my Haswell the RESOURCE_STALLS.SB row is clean as a whistle:

Missing diagram.

Alt: Screen Shot 2018-04-05 at 17.23.38

Source: <https://nicknash.me/wp-content/uploads/2018/04/screen-shot-2018-04-05-at-17-23-38.png>

Changing things to 43 stores, shows a different story for RESOURCE_STALLS.SB, just as we'd expect:

Missing diagram.

Alt: Screen Shot 2018-04-05 at 17.36.50

Source: <https://nicknash.me/wp-content/uploads/2018/04/screen-shot-2018-04-05-at-17-36-50.png>

So that's a second not-very-surprising (but satisfying!) validation that store buffer entries are indeed being exhausted.

I also checked what [IACA](#) had to say about a loop kernel of 42 vs 43 stores followed by a stream of no-ops, it seems not to simulate enough of the microarchitecture to report anything interesting. Just for completeness, I've committed its [42 store throughput](#) and [pipeline trace](#), as well as its [43 store throughput](#) and [pipeline trace](#). It does allow you to see the 4 no-ops per cycle running in parallel with the stores though.

6 Conclusion

If you'd like to experiment with this yourself, you can find the code I used here: <https://github.com/nicknash/GuessStore>. It's a fairly hilarious / horrible setup, a C# program generates a C++ program with the required assembly instructions inside it!

Lastly, I'll just note that although the above demonstrates the store buffer capacity it doesn't nail down the "re-order bound" of the processor. In practice, Intel's store buffers are *coalescing* — they can eliminate consecutive stores to the same location. This results in a 1 larger re-order bound than store buffer capacity. You can find more details in this [paper](#)